A Generic Algebraic Framework for Preimage Attacks: Application to Reduced-Round Ascon-Xof*

Seungjun Baek[1], Giyoon Kim[2], Yongjin Jeon[3], and Jongsung Kim[4,5]

[1] Department of Financial Information Security, Kookmin University, Republic of Korea
hellosj3@kookmin.ac.kr
[2] Department of IT Convergence and Application Engineering, Pukyong National University, Republic of Korea
gykim@pknu.ac.kr
[3] COSIC, KU Leuven, Belgium
idealtop18@gmail.com
[4] Department of Cyber Security, Kookmin University, Republic of Korea
jskim@kookmin.ac.kr
[5] Department of Information Security, Cryptology, and Mathematics, Kookmin University, Republic of Korea

Abstract

Secure and resilient digital infrastructures increasingly rely on standardized lightweight and sponge-based cryptographic primitives deployed across constrained Internet of Things (IoT) and edge environments. In this paper, we propose a generic algebraic framework for preimage attacks on hash functions. We compute the probability that a matrix encoding the relationship between input variables and linear equations derived from the hash value is solvable, and use this probability to estimate the expected guessing time under the assumption that the output polynomials behave randomly in the attack setting. To validate this assumption, we experimentally evaluate the randomness of the underlying permutations by analyzing the variance and entropy of polynomial distributions across rounds, using Ascon as a case study. We also calculate the required bit operations in detail to obtain precise attack complexities. Applying the proposed framework, we present the first preimage attack on 8-round Ascon-Xof faster than exhaustive search. These results provide practical security-margin insights for standardized trust primitives used in lightweight and resource-constrained digital infrastructures.

1 Introduction

The widespread deployment of IoT and edge devices has substantially expanded the attack surface, making resource-constrained platforms attractive targets for adversaries. Ensuring robust cryptographic protection in such environments is therefore essential for maintaining data confidentiality, integrity, and authenticity across modern digital services. In particular, hash and extendable output functions (XOF) primitives serve as low-level trust anchors for software integrity, device authentication, key derivation, secure update mechanisms, and auditability in cloud-edge-IoT ecosystems. As a result, evaluating their security margin is important not only from a cryptanalytic perspective but also for digital trust in resilient service infrastructures.

These requirements have motivated the standardization of lightweight cryptographic primitives that provide well-understood security while remaining efficient on constrained platforms.

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 28, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

To this end, the National Institute of Standards and Technology (NIST) launched an initiative to standardize lightweight cryptography (LWC), focusing on achieving an optimal balance between security and efficiency in constrained environments. As a result of this effort, the Ascon [12] family was selected as the new LWC standard. Ascon includes authenticated encryption schemes (Ascon-128, Ascon-128a, and Ascon-80pq), hash functions (Ascon-Hash and Ascon-Hasha), and extendable output functions (Ascon-Xof and Ascon-Xofa). Detailed specifications are provided in the final NIST Special Publication (SP) 800-232 [35].

The cryptographic community has extensively analyzed Ascon, particularly focusing on the analysis of its underlying permutation [1,8–10,15–17,19,22,28,36,39] and its authenticated encryption schemes [3, 16, 20, 21, 24, 25, 27, 30, 34]. In the context of hashing, the designers of Ascon initially demonstrated that a 2-round permutation can be linearized to recover preimages in reduced-round Ascon-Xof [11]. They also noted that bounding the algebraic degree of reduced-round permutations can accelerate brute-force preimage search when combined with large memory, as suggested by Bernstein [4]. Several follow-up works have explored collision and preimage attacks on reduced-round hashing modes. Zong et al. [41] presented collision attacks on 2-round Ascon-Hash and Ascon-Xof, with the latter shown to be vulnerable in practice. Gerault et al. [16] improved the collision attacks on Ascon-Hash, further advanced by Yu et al. [40]. Qin et al. [32] proposed meet-in-the-middle preimage attacks on 3- and 4-round Ascon-Xof with a 128-bit output, later improved by Li et al. [23] using an SAT-based automatic framework via linearize-and-guess techniques. Baek et al. [2] refined the initial attack of Dobraunig et al. and extended the analysis to the linearizability of 3- and 4-round permutations. More recently, Niu et al. [31] proposed differential-linear preimage attacks on 4-round Ascon-Hash and Ascon-Xof. To date, preimage attacks on Ascon-Xof with five or more rounds remain an unresolved and challenging problem.

XOFs are a class of sponge-based hash primitives that, unlike fixed-length hashes, can generate an arbitrarily long stream of output bits. SHAKE128 and SHAKE256 [14] were the first XOFs standardized by NIST. NIST SP 800-232 [35] further expands this roster by including Ascon-Xof128 (and its variants). This broad output capability renders XOFs essential for key derivation, generating stream cipher keystreams, and creating masks in cryptographic schemes such as RSA-OAEP and the DRIP protocol [37]. Ascon-Xof enjoys widespread support: it is implemented in lightweight stacks such as wolfSSL [38] and CAST's ASCON-F IP cores [7]. Given its broad adoption, rigorous cryptanalysis is essential to ensure its robustness across a wide range of deployment scenarios.

Our Contributions. In this paper, we present a generic algebraic framework for conducting preimage attacks on hash functions and apply it to an important instance: Ascon-Xof. Our main contributions are as follows:

1. **Generic algebraic framework for preimage attacks on hash functions**

   We introduce a generic algebraic framework for preimage attacks on hash functions. The method guesses most of the input bits, leaving only a few as algebraically solvable variables. From the hash value, we derive polynomials, from which a solution can be obtained via Gaussian elimination. We rigorously analyze the probability that the matrix representing the relationship between the input variables and the output polynomials, derived from the hash value, is solvable, and link this to the expected guessing time for the attack. We further analyze the computational complexity of solving the mixed bit-variable state, which we refer to as the *constructing time*, and explicitly calculate the number of bit operations required for Gaussian elimination, thereby quantifying the *solving time*.

We focus in particular on Ascon, as it has been the subject of active research on algebraic preimage attacks. The proposed attacks rely on the assumption that the distribution of the polynomials derived from the hash value is uniform, making them inapplicable to overly reduced rounds. Thus, we experimentally verify that the output polynomial distribution of Ascon becomes uniform only after a certain number of rounds. This validation is performed by analyzing the variance and measuring the entropy of the polynomial distributions.

2. **Preimage attacks on reduced-round Ascon-Xof up to 8 rounds beyond the generic bound**

   Leveraging the proposed framework, we present, for the first time, a preimage attack faster than exhaustive search on up to 8 rounds of Ascon-Xof. To precisely evaluate the constructing time and solving time required for the attack, we compute the number of bit operations based on the number of variables involved in each primitive. This enables us to determine the maximum number of rounds that can be attacked beyond the generic bound. While our attack is evaluated in terms of bit operations with negligible memory usage, making it suitable for practical scenarios, the approach in [4] also uses bit-operation-based evaluation but relies on substantial memory, rendering it less practical.

Organization. Sect. 2 describes the specification of Ascon. Sect. 3 proposes a generic framework for preimage attacks on hash functions. Sect. 4 applies the proposed framework to Ascon-Xof. Sect. 5 provides a discussion on our generic attack presented in Sections 3 and 4. Sect. 6 concludes the paper.

2 Description of Ascon

We provide a brief overview of the Ascon specification as relevant to our analysis.

Ascon [12], designed by Dobraunig et al., includes permutation-based AEAD and hashing schemes. Both schemes are built on 320-bit permutations p^a and p^b with a and b rounds, respectively. In hashing mode, both a and b are set to 12, and the sponge construction [5] is employed (see Figure 1). The 320-bit internal state is divided into five 64-bit words x_i ($0 \leq i \leq 4$), where x_0 is the MSB and x_4 the LSB, as illustrated in Figure 2.

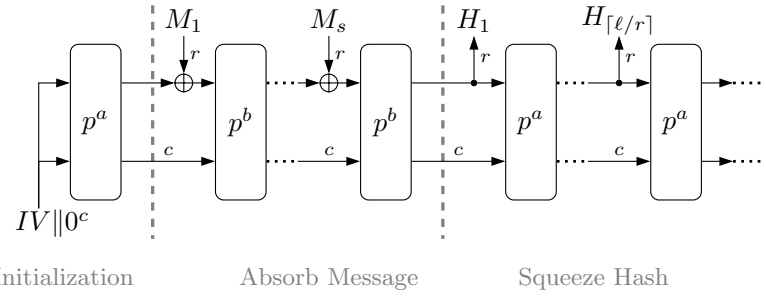


Figure 1: Hashing modes of Ascon [13].

The IV of Ascon is instantiated differently depending on the primitive in use. For Ascon-Xof, the IV is set to `0x00400C0000000000`, after which the initial state is computed as

Table 1: Comparison of preimage attacks on Ascon-Xof. GAF: Generic Algebraic Framework, MitM: Meet-in-the-middle attack, DL: Differential-linear attack, Lin.: Linear structure-based attack, MSR: Multi-step recovery technique-based attack.

Target	Rounds	Hash	Time	Memory (in bits)	Method	Reference
Ascon-Xof	2	64	2^{39}	-	Algebraic†	[11]
			2^{34}	-	Algebraic‡	[2]
			$2^{31.56}$	-	Lin.	[23]
		128	2^{103}	-	Algebraic†	[11]
			2^{98}	-	Algebraic‡	[2]
	3	64	$\mathbf{2^{62.79}}$	-	**GAF**	**Sect. 4**
			2^{56}	-	Algebraic†	[2]
		128	$\mathbf{2^{126.79}}$	-	**GAF**	**Sect. 4**
			$2^{120.58}$	2^{39}	MitM	[32]
			$2^{120.02}$	–	DL	[31]
			2^{120}	-	Algebraic†	[2]
			$2^{114.53}$	2^{30}	MitM	[33]
			$2^{112.21}$	-	Lin.	[23]
	4	64	$\mathbf{2^{63.02}}$	-	**GAF**	**Sect. 4**
			2^{63}	-	Algebraic†	[2]
		128	$\mathbf{2^{127.02}}$	-	**GAF**	**Sect. 4**
			2^{127}	-	Algebraic†	[2]
			$2^{125.47}$	-	DL	[31]
			$2^{124.67}$	2^{54}	MitM	[32]
			$2^{124.49}$	-	Lin.	[23]
	6	64	$2^{63.3}$	2^{67}	Algebraic	[4]
			$\mathbf{2^{63.83}}$	-	**GAF**	**Sect. 4**
	8*	64	$\mathbf{2^{63.98}}$	-	**GAF**	**Sect. 4**
		128	$\mathbf{2^{127.98}}$	-	**GAF**	**Sect. 4**

† A setting considering Ascon's permutation without round constants and initialization, with $IV = 0$.
‡ A setting considering Ascon's permutation without initialization, with $IV = 0$.
∗ With the proposed GAF, attacks with lower preimage complexity than generic bounds are possible for up to 8 rounds (out of 12) of Ascon-Xof.

$S = p^a\left(IV\|0^{256}\right)$. For the construction of IV and the other values used in different Ascon primitives, refer to [12].

Ascon's hashing mode processes the input message M in blocks of r bits. The padding process first appends a single 1 and then the smallest number of zeroes to M such that the length of the padded message is a multiple of r bits. The padded message is then divided into r-bit blocks $M_1, \ldots, M_s$, where $M\|1\|0^{r-1-(|M| \bmod r)}$ ensures the total length is a multiple of r.

The core permutation p is composed of three sequential operations: constant addition p_C, substitution p_S, and linear diffusion p_L ($p = p_L \circ p_S \circ p_C$). To denote a specific round, the round index is added as a subscript. For example, $p_{S,1}$ refers to the substitution layer in round

1. The input state at round R is represented as: $x^R = x_0^R \| x_1^R \| x_2^R \| x_3^R \| x_4^R$, and the output of p_S is denoted by $y^R = y_0^R \| y_1^R \| y_2^R \| y_3^R \| y_4^R$. We denote a single bit of each word by $[\cdot]$; for instance, $x_i^R[j]$ represents the j-th bit (from the left[1]) of word i in round R for $j = 0, \ldots, 63$. A range of bits is written as $x_i^R[j, \ldots, k]$ for $0 \leq j < k \leq 63$.

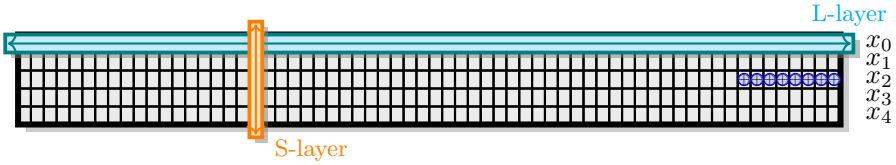


Figure 2: The column-wise S-box layer, the row-wise linear layer and the constant addition.

Addition of Constants (p_C). An 8-bit constant, which varies depending on the round, is added to the $x_2[56, \ldots, 63]$ at each round.

Substitution Layer (p_S). Each column of the 320-bit state is updated by applying Ascon's 5-bit S-box. The algebraic normal form (ANF) of the S-box is given in Figure 3. As the algebraic degree of Ascon's 5-bit S-box is 2, the overall algebraic degree of a single round of permutation p is also 2.

$$
\begin{aligned}
y_0 &= x_4x_1 \oplus x_3 \oplus x_2x_1 \oplus x_2 \oplus x_1x_0 \oplus x_1 \oplus x_0 \\
y_1 &= x_4 \oplus x_3x_2 \oplus x_3x_1 \oplus x_3 \oplus x_2x_1 \oplus x_2 \oplus x_1 \oplus x_0 \\
y_2 &= x_4x_3 \oplus x_4 \oplus x_2 \oplus x_1 \oplus 1 \\
y_3 &= x_4x_0 \oplus x_4 \oplus x_3x_0 \oplus x_3 \oplus x_2 \oplus x_1 \oplus x_0 \\
y_4 &= x_4x_1 \oplus x_4 \oplus x_3 \oplus x_1x_0 \oplus x_1
\end{aligned}
$$

Figure 3: ANF of Ascon's S-box.

Linear Diffusion Layer (p_L). Each row of the 320-bit state is internally diffused by a linear transformation $\sum_i$, where $\sum_i$ is given in Figure 4. Here, $\ggg$ is the right cyclic shift operation over a 64-bit word.

3 A Generic Algebraic Framework for Preimage Attacks on Hash Functions

We describe a new generic preimage attack framework based on the independence of polynomials derived from the hash value. We are particularly interested in applying this framework to Ascon, which has received significant attention in algebraic preimage attacks. Although the framework is generic and applicable to any hash function, we focus on sponge-based hash

[1] In the original Ascon paper [12], bit indices are counted from the right.

$$
\begin{aligned}
x_0 &= \Sigma_0(y_0) = y_0 \oplus (y_0 \ggg 19) \oplus (y_0 \ggg 28) \\
x_1 &= \Sigma_1(y_1) = y_1 \oplus (y_1 \ggg 61) \oplus (y_1 \ggg 39) \\
x_2 &= \Sigma_2(y_2) = y_2 \oplus (y_2 \ggg 1) \oplus (y_2 \ggg 6) \\
x_3 &= \Sigma_3(y_3) = y_3 \oplus (y_3 \ggg 10) \oplus (y_3 \ggg 17) \\
x_4 &= \Sigma_4(y_4) = y_4 \oplus (y_4 \ggg 7) \oplus (y_4 \ggg 41)
\end{aligned}
$$

Figure 4: ANF of Ascon's diffusion layer.

functions, as Ascon belongs to this class. The framework is generic in the following sense. It requires only (i) output-derived polynomial equations over $GF(2)$, (ii) a choice of symbolic input variables, and (iii) a solvability and consistency test for the resulting linearized monomial system. The probability analysis in Sect. 3.2 does not use any Ascon-specific structural property; Ascon-specific details enter only when generating the output polynomials and estimating the construction cost in Sect. 4. Thus, any hash or XOF primitive for which such equations can be generated can be analyzed using the same probability and complexity framework.

3.1 The Framework

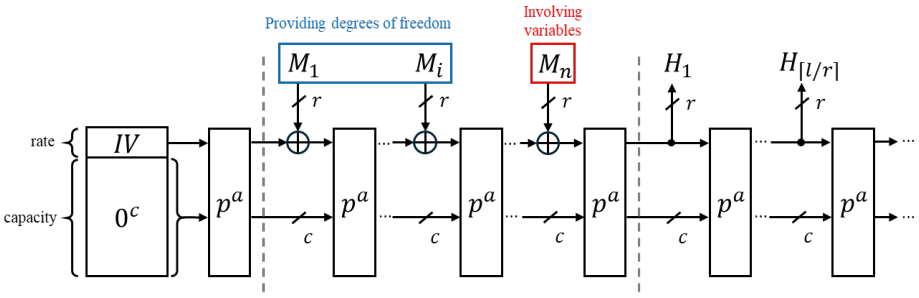


Figure 5: Generic preimage attack framework on sponge-based hash functions.

When the number of unknown variables is denoted by l, the total number of distinct monomials that can be constructed from these variables excluding the constant term is $2^l - 1$. The constant term, if present, can be isolated and handled separately. For example, if three bit variables a_1, a_2, a_3 are involved in positions within the last message block, the monomials that can be found in the polynomials derived from the hash value are as follows:

$$
a_1, a_2, a_3, \underbrace{a_1a_2}_{b_1}, \underbrace{a_2a_3}_{b_2}, \underbrace{a_3a_1}_{b_3}, \underbrace{a_1a_2a_3}_{b_4}
$$

Here, we can consider variables composed of two or more original variables as new variables. Consequently, the set of all terms we consider includes a_1, a_2, a_3 as well as b_1, b_2, b_3, b_4. This allows all derived polynomials to be treated as linear forms, and the number of possible polynomials is 2^{2^l-1}.

In the above example, if the underlying permutation is reduced-round, it is likely that the coefficient of each monomial is not uniformly distributed between 0 and 1. For instance, in the

case of 2-round Ascon, even with a_1, a_2, a_3 present in the last message block M_n, the coefficient of the monomial $a_1a_2a_3$ in the derived polynomial always becomes 0 owing to the properties of the underlying permutation. Therefore, we proceed with the attack under the assumption that, as the number of rounds increases, the coefficients of each monomial are expected to become more randomly distributed. Our generic preimage attack framework, presented in Figure 5, proceeds as follows, with the structure of Ascon serving as the basis for illustration.

1. We begin by considering a one-block message preimage attack on message M_1. In this scenario, we involve N variables in the last message block M_1, fix one of the 2^{r-N} possible assignments for the remaining $(r-N)$ bits, and apply the underlying permutation to match the output with the target hash value H_1. This process results in the generation of r linear equations. For example, refer to Tables 5 and 6 in Appendix A, which analyze Ascon's permutation with 12 rounds and $r = 64$.

2. The constants from the constructed linear equations are moved to the right-hand side, forming a matrix M of size $m \times (2^N - 1)$, where m represents the number of linear equations used in the analysis, and satisfies $m \leq r$.

3. Gaussian elimination is then applied to the augmented matrix $[A|\mathbb{b}]$ of size $m \times 2^N$ to determine whether the system is solvable. If no solution exists, we return to Step 1 and retry the process with a new candidate assigned for the $(r - N)$ bits.

4. If no valid preimage is found, we extend the approach to a two-block message (M_1, M_2), using M_1 to introduce additional degrees of freedom. We then repeat the process described in Steps 1 through 3.

5. When dealing with longer hash outputs, more message blocks, M_i, are appended before the final message block, allowing us to construct the preimage accordingly.

If the matrix in Step 3 is solvable, this indicates that all relevant monomials have been fully determined. Consequently, the final message block can be reconstructed, and the complete preimage message is successfully recovered.

3.2 Deeper Analysis for Attack Complexity

The success of the attack described above hinges on whether the matrix—representing the linear relationships between monomials and the polynomials derived from the hash value—is solvable. The attack's overall complexity is determined by the probability that such a matrix admits a solution. A solvable matrix indicates that all corresponding monomials have been determined, thus enabling the reconstruction of the last message block and the finalization of the preimage.

Let p be a prime number and k a positive integer, and define $q = p^k$. While the number of full-rank $n \times n$ matrices over a finite field is well known, counting matrices that are either non-square or not of full rank requires more detailed analysis. From the perspective of solving a system of equations, a solution may still exist even if the associated matrix is not of full rank.

To estimate the success probability of our attack, we only need the binary case. Let $N_{m,n}(k)$ be the number of $m \times n$ matrices over GF(2) with rank k. By the standard rank-counting formula,

$$N_{m,n}(k) = \prod_{i=0}^{k-1} \frac{(2^m - 2^i)(2^n - 2^i)}{(2^k - 2^i)}.$$

Hence, for uniformly random $A \in \mathrm{GF}(2)^{m\times n}$ and $b \in \mathrm{GF}(2)^m$, the probability that $Ax = b$ is solvable is

$$P_{m,n} = 2^{-mn-m} \sum_{k=0}^{v} N_{m,n}(k) 2^k, \qquad v = \min(m, n),$$

since a rank-k matrix has a k-dimensional column space and thus a random right-hand side belongs to it with probability 2^{k-m}.

We begin by introducing the following q-analog.

Definition 1. *Let q be a prime power. For a positive integer x, the **q-analog of** x is defined as*

$$[x]_q = 1 + q + q^2 + \cdots + q^{x-1} = \frac{q^x - 1}{q - 1}.$$

*The **q-factorial** of x is given by*

$$[x]_q! = [1]_q [2]_q \cdots [x]_q.$$

*The **q-binomial coefficient** (also known as the Gaussian binomial coefficient) is defined as*

$$\begin{bmatrix} n \\ k \end{bmatrix}_q = \frac{[n]_q!}{[k]_q! [n-k]_q!}.$$

We now state the following theorem, which provides the number of $m \times n$ matrices of rank k over the finite field $GF(q)$:

Theorem 1 ([29])**.** *The number of $m \times n$ matrices of rank k, where $0 \le k \le \min\{m, n\}$, over the finite field $GF(q)$ is given by:*

$$\begin{bmatrix} m \\ k \end{bmatrix}_q \times \prod_{i=0}^{k-1} \left(q^n - q^i\right) = \begin{bmatrix} n \\ k \end{bmatrix}_q \times \prod_{i=0}^{k-1} \left(q^m - q^i\right).$$

We denote the number of $m \times n$ matrices of rank $k(\le \min\{m, n\})$ over $GF(q)$ as $C_{m,n}(k; q)$. Using the notation, we state the following Lemma:

Lemma 1. *$C_{m,n}(k; q)$ is given by*

$$C_{m,n}(k; q) = q^{\frac{k(k-1)}{2}} \cdot (q-1)^k \cdot \frac{[m]_q! \cdot [n]_q!}{[k]_q! \cdot [m-k]_q! \cdot [n-k]_q!}.$$

Proof.

$$\begin{aligned}
&C_{m,n}(k; q) \\
&= \begin{bmatrix} m \\ k \end{bmatrix}_q \cdot (q^n - 1)(q^n - q) \cdots \left(q^n - q^{k-1}\right) \\
&= \frac{(q^m - 1)(q^m - q) \cdots \left(q^m - q^{k-1}\right)(q^n - 1)(q^n - q) \cdots \left(q^n - q^{k-1}\right)}{\left(q^k - 1\right)\left(q^k - q\right) \cdots \left(q^k - q^{k-1}\right)} \\
&= q^{\frac{k(k-1)}{2}} \cdot (q-1)^k \\
&\cdot \frac{\prod_{i=1}^{k} \left(q^{m-i} + q^{m-i-1} + \cdots + 1\right) \cdot \prod_{i=1}^{k} \left(q^{n-i} + q^{n-i-1} + \cdots + 1\right)}{\prod_{i=1}^{k} \left(q^{k-i} + q^{k-i-1} + \cdots + 1\right)} \\
&= q^{\frac{k(k-1)}{2}} \cdot (q-1)^k \cdot \frac{[m]_q! \cdot [n]_q!}{[k]_q! \cdot [m-k]_q! \cdot [n-k]_q!}.
\end{aligned}$$

□

We immediately obtain the following corollary.

Corollary 1. *The number of full-rank $m \times n$ matrix over the finite field $GF(q)$ is given by*

$$\begin{aligned} C_{m,n}(v;q) &= (q^u - 1)(q^u - q)\cdots\left(q^u - q^{v-1}\right) \\ &= q^{\frac{v(v-1)}{2}} \cdot (q-1)^v \cdot \frac{[u]_q!}{[u-v]_q!}, \end{aligned}$$

where $u = max(m,n)$ and $v = min(m,n)$.

Next, we analyze the probability that the linear equation $A\mathbb{x} = \mathbb{b}$ is solvable.

Theorem 2. *Let A be an $m \times n$ matrix and $\mathbb{b}$ be an $m \times 1$ vector, both defined over a finite field $GF(q)$. Then, the probability that the equation $A\mathbb{x} = \mathbb{b}$ is solvable is given by:*

$$P^q_{m,n} = q^{-uv-m} \cdot \sum_{k=1}^{v} q^{\frac{k(k+1)}{2}} \cdot (q-1)^k \cdot \frac{[u]_q! \cdot [v]_q!}{[k]_q! \cdot [u-k]_q! \cdot [v-k]_q!},$$

where $u = max(m,n)$ and $v = min(m,n)$.

Proof. Let $u = max(m,n)$ and let $v = min(m,n)$. According to the Rouché–Capelli Theorem, if the coefficient matrix A and its augmented matrix $([A|\mathbb{b}])$ have the same rank, the linear system has solutions. To evaluate the probability that this condition holds, we define the following two events:

1. $S_1^k : rank(A) = k(\leq v)$,
2. $S_2 : rank(A) = rank\left([A|\mathbb{b}]\right)$.

Using the conditional probability formula, we have

$$\begin{aligned} P(S_1^k \cap S_2) &= P(S_2) \cdot P(S_1^k|S_2) \\ &= P(S_1^k) \cdot P(S_2|S_1^k). \end{aligned}$$

We compute $P(S_1^k)$ and $P(S_2|S_1^k)$ to simplify the calculation. Since $P(S_1^k)$ represents the proportion of matrices with rank k among all matrices,

$$P(S_1^k) = \frac{C_{m,n}(k;q)}{q^{uv}} \tag{1}$$

holds by Theorem 1. To compute $P(S_2|S_1^k)$, we consider the probability that the vector $\mathbb{b}$ lies in the column space of matrix A, assuming A has rank k. Since the column space of A has k basis vectors, we have

$$P(S_2|S_1^k) = \frac{q^k}{q^m}. \tag{2}$$

By combining (1) and (2) and summing over all possible cases, we have

$$\begin{aligned}\sum_{k=1}^{v} P(S_1^k \cap S_2) &= \sum_{k=1}^{v} P(S_1^k) \times P(S_2|S_1^k) \\ &= \sum_{k=1}^{v} \frac{C_{m,n}(k;q)}{q^{uv}} \times \frac{q^k}{q^m} \\ &= \sum_{k=1}^{v} \Bigg\{ q^{\frac{k(k-1)}{2}} \cdot (q-1)^k \\ &\quad \cdot \frac{[u]_q! \cdot [v]_q!}{[k]_q! \cdot [u-k]_q! \cdot [v-k]_q!} \cdot q^{k-uv-m} \Bigg\} \\ &= q^{-uv-m} \cdot \sum_{k=1}^{v} \Bigg\{ q^{\frac{k(k+1)}{2}} \cdot (q-1)^k \\ &\quad \cdot \frac{[u]_q! \cdot [v]_q!}{[k]_q! \cdot [u-k]_q! \cdot [v-k]_q!} \Bigg\}.\end{aligned}$$

This concludes the proof. □

Since the argument depends on computing the probability that $\mathbb{b}$ lies in the column space of A, defined over $\mathbb{F}_q^m$, it assumes an underdetermined system where $m \leq n$. In our case, the probability that the equation $A\mathbb{x} = \mathbb{b}$ is solvable over $GF(2)$ is established by the following corollary.

Corollary 2. *Let A be an $m \times n$ matrix and $\mathbb{b}$ be an $m \times 1$ vector, both over $GF(2)$. Then, the probability that the equation $A\mathbb{x} = \mathbb{b}$ is solvable is given by*

$$P_{m,n}^2 = 2^{-uv-m} \cdot \sum_{k=1}^{v} 2^{\frac{k(k+1)}{2}} \cdot \frac{[u]_2! \cdot [v]_2!}{[k]_2! \cdot [u-k]_2! \cdot [v-k]_2!}.$$

As an example, using Corollary 2, consider a system of 64 randomly selected linear equations, each involving three variables and $2^3 - 1$ monomials, with $\mathbb{b}$ as an $m \times 1$ vector. For $m = 64$ and $n = 7$, the solvability probability is computed as follows:

$$\begin{aligned}P_{64,7}^2 &= 2^{-64 \cdot 8} \cdot \sum_{k=1}^{7} 2^{\frac{k(k+1)}{2}} \cdot \frac{[64]_2! \cdot [7]_2!}{[k]_2! \cdot [64-k]_2! \cdot [7-k]_2!} \\ &\approx 2^{-57}.\end{aligned} \tag{3}$$

Table 2 presents the calculated probabilities corresponding to Equation (3), where the probability from Corollary 2 is incorporated into the value for the full-rank case of the 64×7 matrix M.

Correcting the Probability of Contradiction. The previously calculated solvability probability of the matrix system is incomplete. Consider a scenario where an attacker solves the equation $A\mathbb{x} = \mathbb{b}$ using a specific guess and successfully derives all monomials. Despite this, the

Table 2: Probabilities of a 64×7 matrix achieving rank k ($1 \leq k \leq 7$).

k	$P(\text{rank}(M) = k)$	$\log_2(P(\text{rank}(M) = k))$
1	$3.494585 \times 10^{-133}$	-440.0113
2	$2.707476 \times 10^{-112}$	-370.6190
3	4.423622×10^{-92}	-303.4722
4	1.632029×10^{-72}	-238.4722
5	1.359608×10^{-53}	-175.6190
6	2.388605×10^{-35}	-115.0113
7	6.938894×10^{-18}	-57.0

solution may still be invalid due to inherent contradictions. For instance, let $l = 3$ and consider the following assignment:

$$x_1 = 1, x_2 = 0, x_3 = 1,$$
$$x_1x_2 = 0, x_2x_3 = 1, x_3x_1 = 1,$$
$$x_1x_2x_3 = 1.$$

This solution is contradictory. Given that $x_2 = 0$ and $x_3x_1 = 1$, it logically follows that $x_1x_2x_3 = 0$; however, it is recorded as 1, indicating an inconsistency. Such contradictions arise when higher-degree terms (e.g., $x_1x_2, x_2x_3, x_3x_1, x_1x_2x_3$) are treated as independent variables while solving the system. Therefore, the solvability probability must be adjusted to account for these inconsistencies. To formalize this, consider the following theorem.

Theorem 3. *Let N be the number of binary variables $a_1, a_2, \ldots, a_N$. Assuming all monomials constructed from these variables are independently assigned random values, the probability that they are consistent (i.e., contain no contradictions) is given by*

$$P_N = \frac{2^N}{2^{2^N-1}} = 2^{N-2^N+1}.$$

Proof. As we consider N variables, the total number of non-constant monomials (distinct monomials of these variables) is $2^N - 1$. Thus, there are 2^{2^N-1} possible truth configurations across all monomials.

To ensure that all monomials are consistent, it suffices to consider only the N variables $a_1, a_2, \ldots, a_N$, which leads to 2^N possible cases. This completes the proof. □

By considering Corollary 2 and Theorem 3, we derived the following corollary.

Corollary 3. *Let N be the number of binary variables $a_1, a_2, \ldots, a_N$, and let A be a $(m \times 2^N - 1)$ matrix over $GF(2)$, where each row is randomly generated and corresponds to the coefficients of all possible monomials. For a vector $\mathbb{b}$ of size $m \times 1$ over $GF(2)$, the probability that the system $A\mathbb{x} = \mathbb{b}$ is both solvable and free of contradictions among the monomials is:*

$$P = P^2_{m,2^N-1} \cdot P_N = 2^{-uv-m} \cdot \sum_{k=1}^{v} \Bigg\{ 2^{\frac{k(k+1)}{2}} \cdot \frac{[u]_2! \cdot [v]_2!}{[k]_2! \cdot [u-k]_2! \cdot [v-k]_2!} \cdot 2^{N-2^N+1} \Bigg\},$$

where $u = max(m, 2^N - 1)$ and $v = min(m, 2^N - 1)$.

As an example, when $m = 64$ and N is adjusted from 1 to 6, applying Corollary 3 results in Table 3.

Table 3: Probabilities for solvability of $m \times 2^N - 1$ matrix over $GF(2)$ for varying N.

m	N	$\log_2(P_{64,2^N-1})$	$\log_2(P_N)$	$\log_2(P_{64,2^N-1} \times P_N)$
64	1	-63.0	0.0	-63.0
64	2	-61.0	-1.0	-62.0
64	3	-57.0	-4.0	-61.0
64	4	-49.0	-11.0	-60.0
64	5	-33	-26.0	-59
64	6	-1.359	-57.0	-58.359

Finally, this attack relies on the expectation that the distribution of the polynomials derived from the hash value is uniform, making it inapplicable to overly reduced rounds. We demonstrate this using Ascon's permutation. Here, we specifically present the case for Ascon with $N = 3$. In the last message block, we involve three variables in input bit positions 60–62 and analyze the distribution of the output polynomials. For each round, from 2 to the full 12 rounds, at least 200,000,000 ($\approx 2^{27.57}$) random values are substituted into bit positions 0–59. The distribution of the output polynomials is then evaluated according to the variance (in log scale) and Shannon entropy. Figure 6 reveals that for five or more rounds, the polynomials are uniformly distributed.

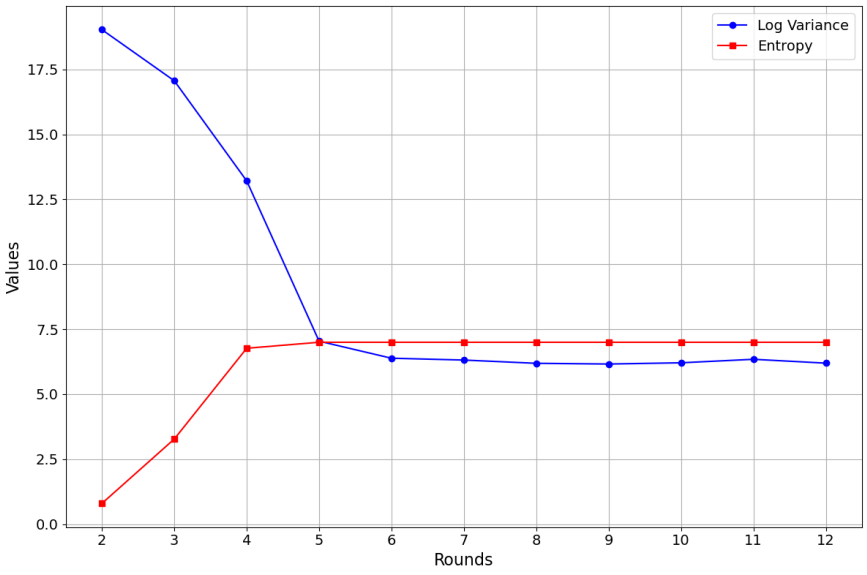


Figure 6: Evaluation of uniformity in output polynomials for Ascon's permutation.

4 Applying the Framework to Reduced-Round Ascon-Xof

Since our attack methodology relies on solving a large system of linear equations, the associated computational cost is considerable and cannot be overlooked. However, evaluating complexity solely in terms of encryption operations does not provide a fair comparison with exhaustive search methods due to the fundamental differences in operations involved. To address this,

we assess the algorithm's complexity by counting fundamental bitwise operations—such as XOR, OR, and AND—performed on pairs of bits. Our goal is to quantify precisely the number of bit operations required to solve the linear expressions generated after a given number of permutation rounds, denotes as R.

When N variables are involved in the last message block, we construct an $m \times 2^N$ matrix that encapsulates the $(2^N - 1)$ monomial coefficients along with the constant term vector $\mathbb{b}$ derived from H_1. To determine whether the system is solvable, we verify if rank(A)=rank($[A|\mathbb{b}]$), which requires computing the row echelon form of the matrix. To simplify notation, we denote the matrix as an $m \times n$ binary matrix, and count the bit operations required for Gaussian elimination. Let $M \stackrel{def}{=} [A|\mathbb{b}]$, where $M[i][j]$ denotes the element in the ith row and jth column, and $M[i][*]$ denotes ith row. In this context, Algorithm 1 details the exact number of bit operations needed to perform Gaussian elimination on a binary matrix.

Algorithm 1 Gaussian elimination algorithm for binary matrices

Input: $(m \times n)$ matrix M $(m \geq n)$
Output: RREF form of matrix M
  for $i = 0, 1, \cdots, m-2$ **do**
    for $j = i+1, i+2, \cdots, m-1$ **do**
      Swap$(M[i][*], M[j \cdot M[j][i] + i \cdot (1 - M[j][i])][*])$
    for $j = i+1, i+2, \cdots, m-1$ **do**
      istrue $= M[i][i] \cdot M[j][i]$
      for $k = i, i+1, \cdots, n-1$ **do**
        $M[j][k] = M[j][k] \oplus (M[i][k] \cdot \mathsf{istrue})$
  return M

Based on Algorithm 1, we calculated the number of bit operations required to perform Gaussian elimination on an $m \times n$ matrix, denoted by T. This complexity can be expressed as:

$$\begin{aligned} T = &\{(2n+1)(m-1) + (2(n-1)-1)(m-2) \\ &+ (2(n-2)-1)(m-3) + \cdots \\ &+ (2 \cdot 2 + 1) \cdot (m-(n-1))\} \\ &+ \lceil \log_2 n \rceil \cdot \{(m-1) + (m-2) + \cdots + (m-n+1)\} \\ = &\sum_{k=1}^{n-1} (2k+3)(m-n+k) + \lceil \log_2 n \rceil \frac{(n-1)(2m-n-1)}{2} \\ = &\, 2 \cdot \frac{n(n-1)(2n-1)}{6} + (2m-2n+3)\frac{n(n-1)}{2} + \\ &+ 3(m-n)(n-1) + \lceil \log_2 n \rceil \frac{(n-1)(2m-n-1)}{2} \\ = &\, \frac{n(n-1)(6m-2n+7)}{6} + 3(m-n)(n-1) \\ &+ \lceil \log_2 n \rceil \frac{(n-1)(2m-n-1)}{2}, \end{aligned} \tag{4}$$

where $m \geq n$.

This includes the row swapping process to create leading ones and all necessary XOR and AND bit operations. In our applications, we present the ratio of the time required to solve the equation system to the time required to perform the underlying permutation using Equation (4).

4.1 Bitwise Operation Complexity in Mixed Bit-Variable States Used in Ascon

We begin by defining a mixed bit-variable state, denoted by Ascon_{poly}, used for computing output polynomials in the context of Ascon. We then analyze the bit-level complexity of the corresponding bitwise operations under this state. The polynomials defined over N-bit variables are presented as:

$$A = \sum_{I \subset \{1,\cdots,N\}} A_I a_I, \qquad B = \sum_{I \subset \{1,\cdots,N\}} B_I a_I,$$

where A and B are the inputs to the operation. Then, the XOR operation between two polynomials is performed as follows:

$$\begin{aligned} A \oplus B &= \sum_{I \subset \{1,\cdots,N\}} A_I a_I \oplus \sum_{I \subset \{1,\cdots,N\}} B_I a_I \\ &= \sum_{I \subset \{1,\cdots,N\}} (A_I \oplus B_I) a_I \end{aligned}$$

Since each coefficient bit A_I and B_I can be computed independently, the XOR operation across the entire polynomial requires at most 2^N XOR operations.

In contrast, the AND operation between two polynomials involves a more intricate process due to the interaction between monomial terms.

$$\begin{aligned} A \cdot B &= \left(\sum_{I \subset \{1,\cdots,N\}} A_I a_I \right) \cdot \left(\sum_{I \subset \{1,\cdots,N\}} B_I a_I \right) \\ &= \sum_{I \subset \{1,\cdots,N\}} \left(\sum_{I_0 \cup I_1 = I} A_{I_0} \cdot B_{I_1} \right) a_I. \end{aligned} \tag{5}$$

According to Equation (5), using a naive approach to obtain the coefficient of each a_I indexed by I requires the following number of bitwise operations:

$$\begin{aligned} &\#(\text{Bit Operation}) \\ &= \underbrace{\binom{N}{0} + \binom{N}{1} \cdot 3^1 \cdots + \binom{N}{N} \cdot 3^N}_{\text{AND operations}} \\ &+ \underbrace{\binom{N}{1} \cdot (3^1 - 1) + \binom{N}{2} \cdot (3^2 - 1) + \cdots + \binom{N}{N} \cdot (3^N - 1)}_{\text{XOR operations}} \\ &= \binom{N}{0} + \binom{N}{1} \cdot (2 \cdot 3^1 - 1) + \cdots + \binom{N}{N} \cdot (2 \cdot 3^N - 1). \end{aligned}$$

However, by employing a technique that reuses intermediate results, the total number of required

bitwise operations can be reduced as follows:

$$\#(\text{Reduced Bit Operation}) = \underbrace{2^N \cdot 1}_{\text{AND operations}} + \underbrace{3 \cdot \sum_{k=1}^{N} 2^{k-1}}_{\text{XOR operations}} = 2^{N+2} - 3.$$

Considering that Ascon has a predetermined initial state, many of its operations can be efficiently handled using single bitwise operations instead of full polynomial computations. This enables a more accurate estimation of the required bitwise operations. By identifying optimal bit positions for variable involvement based on these counts, the attack process can be further optimized.

4.2 Attacks on Ascon

A system of 64 linear equations can be derived from the Ascon permutation based on H_1. For 64-bit hashing, if the resulting system of equations is solvable, a solution can be efficiently obtained via Gaussian elimination. However, for 128-bit hashing, where the hash value is $H_1||H_2$, even if an input message satisfying H_1 is found, it may not match with H_2. Therefore, the probability of randomly obtaining a correct H_2 is approximately 2^{-64}. When the number of degrees of freedom is insufficient, additional messages M_i can be introduced to compensate.

We define constructing time as the time required to compute the derived polynomials using the mixed bit-variable state and guessed values through the Ascon permutation. It is measured in terms of bit operations and denoted by $T_C^{\mathcal{B}}$, which is expressed by dividing the total number of bit operations required for construction by the number of bit operations of the target round of Ascon. As the number of variables involved, N, increases, $T_C^{\mathcal{B}}$ grows due to the rising cost of evaluating more complex polynomials. The matrix $[A|\mathbb{b}]$, which has dimensions 64×2^N, directly influences the computational cost of the Gaussian elimination process. Equation (4) is used to calculate the solving time for each guess, where the number of bit operations required for Gaussian elimination is divided by the total bit operations for the target round of Ascon, and the result is denoted by $T_S^{\mathcal{B}}$. For a single round of Ascon, the total number of bit operations is 2056, which includes $64 \times 22 = 1408$ operations for the substitution layer, 2×320 operations for the linear diffusion layer, and 8 operations for constant addition. In the final round, only the top row needs to be computed, reducing the required number of bit operations to 208. Using these values, we calculate the constructing time, guessing time, and overall attack complexity required to recover the preimage of a 64-bit hash. These are detailed in Table 4 and illustrated in Figure 7. Notably, attack complexity is generally higher at lower rounds, as it is larger due to the higher relative solving cost compared to the total bit operations for a given round of Ascon-Xof. For 128-bit hashing with Ascon-Xof, the total complexity is scaled by 2^{64} to account for the double output length. Appendix B presents the optimal bit indices for variable selection that yield the lowest constructing time, leveraging the known fixed initial state of Ascon.

5 Discussion

This section discusses the *constructing time*, which refers to the cost of generating equations from the mixed bit-variable state. In the initial version of linearization-based preimage attacks [18], the emphasis was primarily on the number of guesses. The total complexity was

Table 4: Comparison of attack complexity for Ascon-Xof based on rounds and N.

m	R	N	$\#G$	$T_C^{\mathcal{B}} + T_S^{\mathcal{B}}$	$T^{\mathcal{B}}$
64	3	1	2^{63}	$2^{-0.02} + 2^{-2.93}$	$2^{63.16}$
		2	2^{62}	$2^{0.13} + 2^{-0.63}$	$2^{62.80}$
		3	2^{61}	$2^{0.27} + 2^{1.16}$	$2^{62.79}$
		4	2^{60}	$2^{0.42} + 2^{2.79}$	$2^{63.04}$
	4	1	2^{63}	$2^{0.34} + 2^{-3.50}$	$2^{63.44}$
		2	2^{62}	$2^{0.82} + 2^{-1.20}$	$2^{63.14}$
		3	2^{61}	$2^{1.35} + 2^{0.60}$	$2^{63.02}$
		4	2^{60}	$2^{1.90} + 2^{2.23}$	$2^{63.07}$
	5	1	2^{63}	$2^{0.63} + 2^{-3.90}$	$2^{63.69}$
		2	2^{62}	$2^{1.41} + 2^{-1.60}$	$2^{63.58}$
		3	2^{61}	$2^{2.27} + 2^{0.20}$	$2^{63.58}$
		4	2^{60}	$2^{3.15} + 2^{1.82}$	$2^{63.64}$
	6	1	2^{63}	$2^{0.79} + 2^{-4.21}$	$2^{63.83}$
		2	2^{62}	$2^{1.72} + 2^{-1.91}$	$2^{63.83}$
		3	2^{61}	$2^{2.71} + 2^{-0.12}$	$2^{63.90}$
	7	1	2^{63}	$2^{0.88} + 2^{-4.47}$	$2^{63.92}$
		2	2^{62}	$2^{1.89} + 2^{-2.17}$	$2^{63.97}$
	8	1	2^{63}	$2^{0.95} + 2^{-4.69}$	$2^{63.98}$

$\#G$: Number of guesses
$T_C^{\mathcal{B}}$: Constructing time in terms of bit operations
$T_S^{\mathcal{B}}$: Solving time in terms of bit operations
$T^{\mathcal{B}}$: Attack complexity in terms of bit operations ($\#G \cdot (T_C^{\mathcal{B}} + T_S^{\mathcal{B}})$)

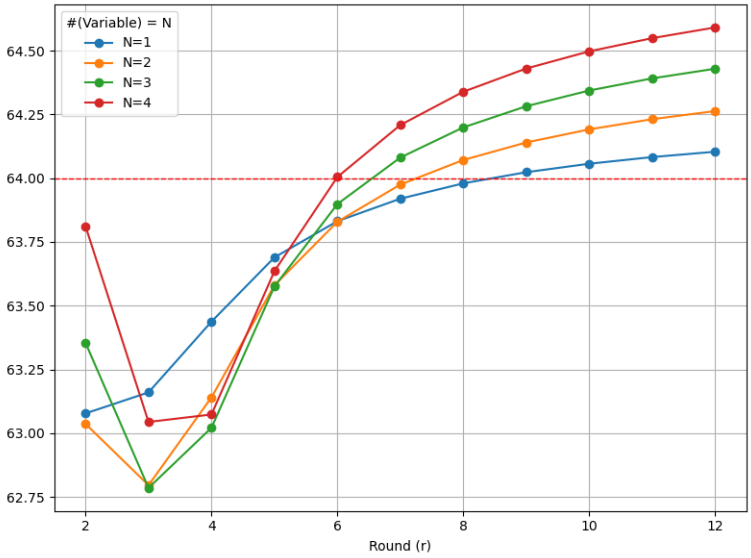


Figure 7: Preimage attack complexity of Ascon-Xof (64-bit output) across rounds and N.

estimated by subtracting the gain from degrees of freedom and the number of linear equations from the complexity of the generic preimage attack. Later, Liu et al. [26] emphasized that the cost of solving the linear equations must be included, accounting for the time required to solve each guess in the overall complexity. This adjustment has since been consistently adopted in related research. However, to the best of our knowledge, the time required to construct the polynomials derived from the hash value has not been adequately addressed in the literature. This omission likely stems from the assumption that constructing the equation system through the underlying permutation is a simple, automated process that does not considerably impact the attack complexity. Yet, in a mixed bit-variable state, equation construction cannot be efficiently modeled using a 1-bit datapath per variable. As a result, the number of bit operations required for this process increases substantially. Ignoring this cost may lead to an underestimation of the overall attack complexity and hinder the provision of proper feedback for secure cipher design. Therefore, a more conservative and accurate complexity evaluation must incorporate this factor. In our study, we include constructing time by comparing the number of bit operations required for Ascon_{poly} to those required for standard operations in Ascon. Although the exact metric may vary depending on the attack environment, this work represents the first to account for this construction overhead explicitly. This analysis may serve as a starting point for further investigation into the equation construction phase in linearization-based preimage attacks.

We adopt Bernstein's bit-operation-based evaluation methodology [4], applied initially to Keccak [6], which defines attack complexity as the ratio of the total number of bit operations required by the attack to those needed for a single execution of the target primitive. While both approaches share this same evaluation goal, they differ technically: our method encodes polynomial coefficients into a matrix and directly counts the required bit operations to solve it, whereas Bernstein's method estimates operations based on a polynomial enumeration algorithm used to search for preimages. A notable challenge in Bernstein's approach is its high memory and communication overhead, which can limit its practical applicability. In contrast, our method involves negligible memory consumption, representing a modest yet meaningful advancement in bit-operation based complexity analysis.

The advantage over the generic 2^{64} bound is numerically small and should not be interpreted as a practical break of full-round Ascon-Xof. Its significance is theoretical and methodological: it is the first reduced-round Ascon-Xof preimage result crossing the generic bound for 8 rounds under negligible-memory assumptions, and it shows where this class of algebraic linearization attacks reaches its limit once equation-construction overhead is counted. Thus, the result is best understood as a security-margin assessment rather than a deployment-level attack.

6 Conclusion and Future Work

In this paper, we presented a generic algebraic framework for preimage attacks based on the independence assumption on polynomials derived from the hash value. We applied this framework to Ascon-Xof and obtained, for the first time, preimage attacks on its 8-round reduced version with complexity slightly below exhaustive search and negligible memory, offering an efficiency advantage over prior memory-intensive approaches. Beyond these specific reduced-round results, the proposed framework provides a practical way to assess the preimage-resistance margin of standardized lightweight hash and XOF primitives in deployment-oriented settings. Extending this framework to other NIST-standardized primitives, such as Keccak-based constructions, would also be an important direction for future work.

Acknowledgments

This work was supported by the IITP(Institute of Information & Communications Technology Planning & Evaluation)-ITRC(Information Technology Research Center) grant funded by the Korea government(Ministry of Science and ICT)(IITP-2026-RS-2022-00164800).

References

[1] Seungjun Baek, Yongjin Jeon, Giyoon Kim, and Jongsung Kim. On impossible and truncated distinguishers for iot-friendly AEAD algorithms. *IEEE Internet Things J.*, 11(19):31878–31891, 2024.

[2] Seungjun Baek, Giyoon Kim, and Jongsung Kim. Preimage attacks on reduced-round ascon-xof. *Des. Codes Cryptogr.*, 92(8):2197–2217, 2024.

[3] Jules Baudrin, Anne Canteaut, and Léo Perrin. Practical cube attack against nonce-misused ascon. *IACR Trans. Symmetric Cryptol.*, 2022(4):120–144, 2022.

[4] J Bernstein, Daniel. Second preimages for 6 (7 (8??)) rounds of keccak? *Posted on the NIST mailing list*, 2010. https://ehash.iaik.tugraz.at/uploads/6/65/NIST-mailing-list_Bernstein-Daemen.txt.

[5] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. In *ECRYPT hash workshop*. Citeseer, 2007. https://csrc.nist.rip/groups/ST/hash/documents/JoanDaemen.pdf.

[6] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Keccak. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 313–314. Springer, 2013.

[7] CAST Inc. Ascon-f lightweight authenticated encryption ip core. https://www.cast-inc.com/security/encryption-primitives/ascon-f. Accessed: July 2025.

[8] Aslí Basak Civek and Cihangir Tezcan. Differential-linear attacks on permutation ciphers revisited: Experiments on ascon and drygascon. In *ICISSP 2022*, pages 202–209. SCITEPRESS, 2022. https://doi.org/10.5220/0010982600003120.

[9] Christoph Dobraunig, Maria Eichlseder, and Florian Mendel. Heuristic tool for linear cryptanalysis with applications to caesar candidates. In *ASIACRYPT 2015*, pages 490–509. Springer, 2015. https://doi.org/10.1007/978-3-662-48800-3_20.

[10] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Cryptanalysis of ascon. In *CT-RSA 2015*, pages 371–387. Springer, 2015. https://doi.org/10.1007/978-3-319-16715-2_20.

[11] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Preliminary analysis of ascon-xof and ascon-hash. *Technique Report*, 2019. https://ascon.iaik.tugraz.at/files/Preliminary_Analysis_of_Ascon-Xof_and_Ascon-Hash_v01.pdf.

[12] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *J. Cryptol.*, 34(3):33, 2021.

[13] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon resources. Accessed Aug 2024. https://ascon.iaik.tugraz.at/resources.html.

[14] Morris Dworkin. Sha-3 standard: Permutation-based hash and extendable-output functions, 2015. https://doi.org/10.6028/NIST.FIPS.202.

[15] Johannes Erlacher, Florian Mendel, and Maria Eichlseder. Bounds for the security of ascon against differential and linear cryptanalysis. *IACR Trans. Symmetric Cryptol.*, 2022(1):64–87, 2022. https://doi.org/10.46586/tosc.v2022.i1.64-87.

[16] David Gerault, Thomas Peyrin, and Quan Quan Tan. Exploring differential-based distinguishers and forgeries for ascon. *IACR Trans. Symmetric Cryptol.*, 2021(3):102–136, 2021. https://doi.

org/10.46586/tosc.v2021.i3.102-136.

[17] Faruk Göloğlu, Vincent Rijmen, and Qingju Wang. On the division property of s-boxes. *Cryptology ePrint Archive*, 2016. http://eprint.iacr.org/2016/188.

[18] Jian Guo, Meicheng Liu, and Ling Song. Linear structures: Applications to cryptanalysis of round-reduced keccak. In *ASIACRYPT 2016*, volume 10031 of *LNCS*, pages 249–274. Springer, 2016.

[19] Solane El Hirch, Silvia Mella, Alireza Mehrdad, and Joan Daemen. Improved differential and linear trail bounds for ASCON. *IACR Trans. Symmetric Cryptol.*, 2022(4):145–178, 2022.

[20] Kai Hu. Improved conditional cube attacks on ascon aeads in nonce-respecting settings with a break-fix strategy. *IACR Trans. Symmetric Cryptol.*, 2024(2):118–140, 2024.

[21] Philipp Jovanovic, Atul Luykx, and Bart Mennink. Beyond 2 c/2 security in sponge-based authenticated encryption modes. In *ASIACRYPT 2014*, pages 85–104. Springer, 2014. https://doi.org/10.1007/978-3-662-45611-8_5.

[22] Gregor Leander, Cihangir Tezcan, and Friedrich Wiemer. Searching for subspace trails and truncated differentials. 2018(1):74–100, 2018. https://doi.org/10.13154/tosc.v2018.i1.74-100.

[23] Huina Li, Le He, Shiyao Chen, Jian Guo, and Weidong Qiu. Automatic preimage attack framework on ascon using a linearize-and-guess approach. *IACR Trans. Symmetric Cryptol.*, 2023(3):74–100, 2023.

[24] Yanbin Li, Guoyan Zhang, Wei Wang, and Meiqin Wang. Cryptanalysis of round-reduced ascon. *Science China Information Sciences*, 60(3):1–2, 2017. https://doi.org/10.1007/s11432-016-0283-3.

[25] Zheng Li, Xiaoyang Dong, and Xiaoyun Wang. Conditional cube attack on round-reduced ascon. *IACR Trans. Symmetric Cryptol.*, 2017(1):175–202, 2017. https://doi.org/10.13154/tosc.v2017.i1.175-202.

[26] Fukang Liu, Takanori Isobe, Willi Meier, and Zhonghao Yang. Algebraic attacks on round-reduced keccak. In Joonsang Baek and Sushmita Ruj, editors, *Information Security and Privacy*, volume 12882 of *Lecture Notes in Computer Science*, pages 91–110, Cham, 2021. Springer International Publishing.

[27] Meicheng Liu, Xiaojuan Lu, and Dongdai Lin. Differential-linear cryptanalysis from an algebraic perspective. In *CRYPTO 2021*, pages 247–277. Springer, 2021. https://doi.org/10.1007/978-3-030-84252-9_9.

[28] Rusydi H Makarim and Raghvendra Rohit. Towards tight differential bounds of ascon: A hybrid usage of smt and milp. *IACR Trans. Symmetric Cryptol.*, 2022(3):303–340, 2022. https://doi.org/10.46586/tosc.v2022.i3.303-340.

[29] Kent E Morrison. Integer sequences and matrices over finite fields. *arXiv preprint math/0606056*, 2006.

[30] Yusuke Naito, Yu Sasaki, and Takeshi Sugawara. Committing security of ascon: Cryptanalysis on primitive and proof on mode. *IACR Trans. Symmetric Cryptol.*, 2023(4):420–451, 2023.

[31] Zhongfeng Niu, Kai Hu, Siwei Sun, Zhiyu Zhang, and Meiqin Wang. Speeding up preimage and key-recovery attacks with highly biased differential-linear approximations. In *CRYPTO 2024*, volume 14923 of *Lecture Notes in Computer Science*, pages 73–104. Springer, 2024.

[32] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In *EUROCRYPT 2023*, volume 14007 of *Lecture Notes in Computer Science*, pages 158–188. Springer, 2023.

[33] Lingyue Qin, Boxin Zhao, Jialiang Hua, Xiaoyang Dong, and Xiaoyun Wang. Weak-diffusion structure: Meet-in-the-middle attacks on sponge-based hashing revisited. *IACR Cryptol. ePrint Arch.*, page 518, 2023.

[34] Raghvendra Rohit, Kai Hu, Sumanta Sarkar, and Siwei Sun. Misuse-free key-recovery and distinguishing attacks on 7-round ascon. *IACR Trans. Symmetric Cryptol.*, 2021(1):130–155, 2021. https://doi.org/10.46586/tosc.v2021.i1.130-155.

[35] Meltem Sönmez Turan, Kerry McKay, Donghoon Chang, Jinkeon Kang, and John Kelsey. Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions. Technical report, NIST Special Publication 800-232, 2025.

[36] Yosuke Todo. Structural evaluation by generalized integral property. In *EUROCRYPT 2015*, pages 287–314. Springer, 2015. https://doi.org/10.1007/978-3-662-46800-5_12.

[37] Adam Wiethuechter, Stuart W. Card, and Robert Moskowitz. DRIP Entity Tag Authentication Formats & Protocols for Broadcast Remote ID. Internet-Draft draft-ietf-drip-auth-29, Internet Engineering Task Force, February 2023. Work in Progress.

[38] wolfSSL Inc. wolfssl embedded ssl library. https://www.wolfssl.com, 2016. Accessed: July 2025.

[39] Hailun Yan, Xuejia Lai, Lei Wang, Yu Yu, and Yiran Xing. New zero-sum distinguishers on full 24-round keccak-f using the division property. *IET Information Security*, 13(5):469–478, 2019. https://doi.org/10.1049/iet-ifs.2018.5263.

[40] Xiaorui Yu, Fukang Liu, Gaoli Wang, Siwei Sun, and Willi Meier. A closer look at the s-box: Deeper analysis of round-reduced ASCON-HASH. In *Selected Areas in Cryptography - SAC 2023 - 30th International Conference, Fredericton, Canada, August 14-18, 2023, Revised Selected Papers*, volume 14201 of *Lecture Notes in Computer Science*, pages 22–42. Springer, 2023.

[41] Rui Zong, Xiaoyang Dong, and Xiaoyun Wang. Collision attacks on round-reduced gimli-hash/ascon-xof/ascon-hash. *IACR Cryptol. ePrint Arch.*, page 1115, 2019. https://eprint.iacr.org/2019/1115,.

Appendix

A Output Polynomials of Full Ascon's Permutation

We present example cases in which the guess for the hash value H_1 : `329ecef2bcd7bb12` was successful and unsuccessful during the preimage search process. The process begins by setting the state after absorbing M to `4153434f4e5f5f21`, which implies that, considering the first row value of the S state is `b57e273b814cd416`, M is `f42d6474cf138b37`. Next, the value obtained after applying the inverse linear layer to H_1 is `0xd23883788f309d7c`, and we observe the corresponding polynomials at that position. Assuming $N = 3$, we introduce variables at indices 35, 41, and 49, which were experimentally identified as the positions requiring the fewest bit operations to compute Ascon_{poly}. Tables 5 and 6 present cases where the guess was successful and unsuccessful, respectively. Specifically, Table 6 indicates that the guess can be identified as a failure due to inconsistency in the equations at l_{49}, which is inherently contradictory; additionally, l_{10} and l_{48} conflict with each other.

B Bit Index Selection for Minimizing Constructing Time in Ascon-Xof

We experimentally identify the bit positions at which N variables should be introduced to minimize the construction time, given the fixed initial state of Ascon-Xof. The experiments are conducted by searching for the combination of bit positions that yields the lowest constructing time for each pair of round R and number of variables N. The results are presented in Table 7, where we define $I^{(R,N)}$ as the set of bit index tuples that achieve the lowest construction time for a given round R and number of variables N. If multiple tuples yield the same minimum constructing time, all such tuples are included in $I^{(R,N)}$.

Table 5: Derived polynomials when the guess succeeds for full Ascon's permutation (with $x_{35} = 0, x_{41} = 1, x_{49} = 1$, matching with 'the state after absorbing M').

The state after absorbing M : 01000001 01010011 01000011 01001111 01x_{35}01110 0x_{41}011111 0x_{49}011111 00100001
$l_0 : 1 = x_{41} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_1 : 1 = 1 \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_2 : 0 = x_{35} \oplus x_{41} \oplus x_{41}x_{49}$
$l_3 : 1 = x_{35} \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_4 : 0 = x_{35} \oplus x_{41} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_5 : 0 = 1 \oplus x_{41}x_{49}$
$l_6 : 1 = x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_7 : 0 = 1 \oplus x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_8 : 0 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}$
$l_9 : 0 = x_{35} \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{10} : 1 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{11} : 1 = x_{35} \oplus x_{41} \oplus x_{49}x_{35}$
$l_{12} : 1 = 1 \oplus x_{35} \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{13} : 0 = 1 \oplus x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{14} : 0 = 1 \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{15} : 0 = x_{35} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{16} : 1 = 1 \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{17} : 0 = x_{35} \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{18} : 0 = x_{35} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{19} : 0 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{41}x_{35}$
$l_{20} : 0 = 1 \oplus x_{35} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{21} : 0 = x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{22} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{49}$
$l_{23} : 1 = 1 \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{24} : 0 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{25} : 1 = x_{41} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{26} : 1 = x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{27} : 1 = x_{35} \oplus x_{41}x_{49}$
$l_{28} : 1 = 1 \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{29} : 0 = x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{30} : 0 = x_{41} \oplus x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{31} : 0 = 1 \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{32} : 1 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{33} : 0 = x_{41} \oplus x_{49} \oplus x_{49}x_{35}$
$l_{34} : 0 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49}$
$l_{35} : 0 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{36} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{49}$
$l_{37} : 1 = x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{38} : 1 = 1 \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{39} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{49}$
$l_{40} : 0 = 1 \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{41} : 0 = x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{42} : 1 = x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{43} : 1 = x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35}$
$l_{44} : 0 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35}$
$l_{45} : 0 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{49}x_{35}$
$l_{46} : 0 = 1 \oplus x_{41} \oplus x_{41}x_{49}x_{35}$
$l_{47} : 0 = 1 \oplus x_{35} \oplus x_{41}$
$l_{48} : 1 = x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{49} : 0 = 1 \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{49}x_{35}$
$l_{50} : 0 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49}$
$l_{51} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{49}$
$l_{52} : 1 = x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{53} : 1 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{54} : 0 = x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{55} : 1 = x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{56} : 0 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49}$
$l_{57} : 1 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{58} : 1 = 1 \oplus x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{59} : 1 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{60} : 1 = x_{41}x_{35} \oplus x_{41}x_{49}$
$l_{61} : 1 = x_{41} \oplus x_{41}x_{35}$
$l_{62} : 0 = x_{35} \oplus x_{49}x_{35}$
$l_{63} : 0 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{41}x_{35}$

Table 6: Derived polynomials when the guess fails for full ASCON's permutation.

The state after absorbing M : 1$1000001$ 01010011 01000011 01001111 $01x_{35}01110$ $0x_{41}011111$ $0x_{49}011111$ 00100001
$l_0 : 1 = x_{35} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}$
$l_1 : 1 = x_{35} \oplus x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_2 : 0 = 1 \oplus x_{41} \oplus x_{41}x_{49}x_{35}$
$l_3 : 1 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_4 : 0 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_5 : 0 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_6 : 1 = x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_7 : 0 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_8 : 0 = x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}$
$l_9 : 0 = x_{35} \oplus x_{41} \oplus x_{41}x_{35}$
$l_{10} : 1 = x_{41} \oplus x_{41}x_{35}$
$l_{11} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{12} : 1 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49}$
$l_{13} : 0 = x_{41} \oplus x_{41}x_{35} \oplus x_{49}x_{35}$
$l_{14} : 0 = 1 \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{15} : 0 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35}$
$l_{16} : 1 = x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{17} : 0 = x_{41} \oplus x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{18} : 0 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35}$
$l_{19} : 0 = 1 \oplus x_{41} \oplus x_{41}x_{35}$
$l_{20} : 0 = 1 \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}$
$l_{21} : 0 = x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{22} : 1 = 1 \oplus x_{41} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{23} : 1 = x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{24} : 0 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{25} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{26} : 1 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{41}x_{35}$
$l_{27} : 1 = x_{35} \oplus x_{41}x_{49}$
$l_{28} : 1 = 1 \oplus x_{35} \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{29} : 0 = x_{35} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{30} : 0 = x_{41}x_{35} \oplus x_{41}x_{49}$
$l_{31} : 0 = 1 \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35}$
$l_{32} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{35}$
$l_{33} : 0 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{34} : 0 = x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{35} : 0 = x_{41} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{36} : 1 = 1 \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{37} : 1 = 1 \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{38} : 1 = x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49}$
$l_{39} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{40} : 0 = x_{41} \oplus x_{49}x_{35}$
$l_{41} : 0 = x_{49} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{42} : 1 = 1 \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{43} : 1 = x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{44} : 0 = x_{35} \oplus x_{49} \oplus x_{41}x_{35}$
$l_{45} : 0 = 1 \oplus x_{35}$
$l_{46} : 0 = 1 \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{47} : 0 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{49}x_{35}$
$l_{48} : 1 = 1 \oplus x_{41} \oplus x_{41}x_{35}$
$l_{49} : 0 = 1$
$l_{50} : 0 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{51} : 1 = x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{52} : 1 = x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{53} : 1 = 1 \oplus x_{41} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{54} : 0 = x_{41} \oplus x_{49} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{55} : 1 = 1 \oplus x_{35} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{41}x_{49}x_{35}$
$l_{56} : 0 = 1 \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{57} : 1 = 1 \oplus x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{58} : 1 = x_{35} \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{59} : 1 = x_{41} \oplus x_{49} \oplus x_{41}x_{35}$
$l_{60} : 1 = x_{35} \oplus x_{41} \oplus x_{49} \oplus x_{41}x_{49}$
$l_{61} : 1 = x_{41}x_{49} \oplus x_{49}x_{35}$
$l_{62} : 0 = x_{35} \oplus x_{41} \oplus x_{41}x_{35} \oplus x_{41}x_{49}x_{35}$
$l_{63} : 0 = x_{49} \oplus x_{41}x_{35} \oplus x_{41}x_{49} \oplus x_{49}x_{35} \oplus x_{41}x_{49}x_{35}$

Table 7: Experimentally selected bit positions minimizing constructing time in Ascon-Xof.

R	N	$\mathcal{I}^{(R,N)}$
3	1	(35), (48), (51), (59), (62)
	2	(47, 62)
	3	(32, 47, 62)
	4	(32, 47, 56, 62)
4	1	(51)
	2	(35, 62)
	3	(32, 47, 62)
	4	(48, 54, 56, 62)
5	1	(51)
	2	(51, 56)
	3	(35, 41, 49), (48, 54, 62)
	4	(48, 54, 56, 62)
6	1	(51)
	2	(51, 56)
	3	(48, 54, 62), (35, 41, 49)
7	1	(51)
	2	(51, 56)
8	1	(51)